When Does Training Produce Length-Robust In-Context Programs?

Michael Murray

August 2025

1 Vision

Language models realize sequence-to-sequence maps on a single evolving sequence buffer, where each token is generated as a function of the entire current prefix, including both externally provided tokens and the model's own past outputs. This is also referred to as in-context learning. For a sequence map to be implementable as an online computation it must be scale-invariant in the following sense: a scale-invariant sequence map is one where the program computing the next token from the prefix has a size independent of the prefix length or content. In this setting, and assuming sufficient representational capacity to encode the necessary prefix information, we say that a model is robust to sequence length if task performance remains stable as the sequence grows. Empirically, however, this property is not satisfied by language models, which exhibit pronounced length-dependent failures even on tasks whose underlying structure is scale-invariant. Extended conversations degrade in coherence despite no increase in required contextual information (the "context rot" phenomenon), and reasoning traces deteriorate as intermediate steps accumulate, even when each step depends only on a fixed summary of prior computation.

A natural question is why such length-dependent failures arise. One possibility is that they reflect architectural or resource limitations, such as finite context windows or insufficient capacity. However, these failures have been documented across a wide range of benchmarks, tasks, and architectures, often arising well within available context windows and persisting in highly overparameterized models. Moreover, similar breakdowns are observed even on simple algorithmic and formal language tasks - such as parity tracking — whose underlying sequence maps are exactly scale-invariant and can be implemented by finite-state automata. An interesting empirical observation is that while modern language models typically perform well within the range of sequence lengths encountered during training, performance degrades sharply outside of this regime. Together, these observations suggest that the central issue is not resource limitation, but the emergence during training of update rules whose behavior depends on absolute prefix length. Understanding why gradient-based training yields such length-coupled update dynamics, rather than procedures that generalize robustly across scale, is a central open problem.

This project aims to develop a mathematical theory of length generalization for in-context learners. While existing work on the theory of in-context learning has largely focused on representational capacity, comparatively little is known concerning how training dynamics determine whether or not the update rules learned by such models during pre-training are robust to sequence length. The central objective is to characterize, for well-defined classes of scale-invariant sequence maps and analytically tractable yet sufficiently expressive model families, when gradient-based training converges to update dynamics whose behavior is independent of absolute prefix length, and when

it provably converges instead to length-coupled heuristics. Accordingly, the project analyzes how optimization dynamics, training data distributions, and curriculum structure jointly determine the update rules learned during training. Establishing when scale-robust update dynamics can and cannot be acquired is essential for understanding the limits of current in-context learners and for enabling reliable long-horizon behavior in settings such as extended reasoning, tool use, planning, and sustained interaction.

2 Approach

As a first step toward understanding when length-robust update rules can be acquired during training, this project studies settings which isolate the role of learning dynamics from representational and task-level confounds. We consider a target in-context computation which corresponds to a well-defined, scale-invariant program, and analyze whether gradient-based training recovers the scale robust solution versus a brittle, length-dependent heuristic. More specifically, on the problem side we restrict our attention to tasks chosen to make the distinction between scale-robust and length-coupled solutions unambiguous.

A. **Unambiguous conditional next-token rule:** The task induces a single, well-defined next output for every prefix, potentially conditioned on task-relevant information encoded in the prefix. This includes settings in which the target computation branches between multiple algorithms (e.g., addition versus multiplication) based on an explicit or implicit task indicator, while preserving a unique continuation at each step. This condition removes distributional ambiguity and allows failures of length generalization to be attributed directly to the learned update rule, rather than a lack of task specification.

B. **Known scale-invariant solution with constant program size:** the task has a solution which can be implemented by a fixed-size update program, or Automaton, applied uniformly across all sequence lengths. This is necessary in order that scale-robust behavior is achievable in principle.

On the model and training side, we study a hierarchy of model restrictions that progressively factor out model expressivity and memory fidelity as confounds, allowing us to isolate—and later reintroduce—these effects in a controlled way.

1. **Model expressivity sufficient to represent the scale-invariant program solution:** the model is chosen so that the scale-robust solution lies within its representational capacity. This ensures that any length generalization failure is not due to insufficient expressivity, but arises due to the training dynamics.

2. **Explicit separation between control logic and information storage:** the update rule is explicitly composed of a controller function, learned during training, and an external data structure (for example a stack or tape), which the controller must interact with. The model therefore parametrizes only the update logic, thereby avoiding data storage confounds, and allows us to interpret the learned controller as a program.

3. **Synchronous input–output updates to avoid confounding variable-time computation:** The model processes one input symbol and produces one output symbol per step, avoiding additional complications arising from variable-time or multi-step internal computation. This allows length dependence in the learned update rule to be studied directly, without confounding effects from learning when to compute rather than what to compute.

The work packages are organized as a staged progression in which both the learning problem and the model assumptions are systematically relaxed while preserving the core conditions (A)–(B). We emphasize that this approach enables two complementary contributions with relevance beyond the models studied here. First, it allows us to establish theoretical limitations of standard, gradient-based training procedures by by identifying regimes in which scale-invariant update rules provably cannot be recovered, even in benign settings where the correct program is simple, unique, and well within the model class. These results clarify which failures of length generalization arise from fundamental optimization bias as opposed to architectural or representational constraints. Second, it provides a principled testbed for developing and evaluating new training methods—covering curriculum design, regularization strategies and architectural constraints—explicitly aimed at suppressing length-coupled heuristics.

2.1 Framework for work packages

To recap, the central objective of this project is to understand when gradient-based training yields update rules whose behavior is robust to sequence length, and when it instead converges to length-coupled heuristics that succeed only within the training horizon. We use *automaton recovery* as a controlled testbed for this question. The key advantage is that the target computation is a fully specified, scale-invariant program: recovering the intended program (up to equivalence) implies genuine length generalization by construction, whereas failures can be attributed directly to properties of the learned update dynamics rather than ambiguity in the task or evaluation.

To keep the project sharply scoped while retaining sufficient richness to exhibit length-dependent failures, we focus on *pushdown transducers (PDTs)* as the target class. PDTs are among the simplest scale-invariant sequence-to-sequence models that require nontrivial state evolution: they strictly extend finite-state transducers by allowing a stack, thereby capturing structured dependencies (e.g. nested structure and unbounded counters) while remaining amenable to mechanistic interpretation and, in several regimes, to rigorous analysis. This choice aims to isolate the emergence of length-coupled update rules in a setting where the correct solution is simple and uniform across time, yet where shortcut strategies that overfit the training horizon can still arise.

Single-stream in-context specification. We emphasize that each training example consists of a *single* prompt containing all task information and all input symbols needed to determine the target continuation. Concretely, prompts are structured as a single binary string with embedded operation symbols (when relevant), followed by an "=" token indicating that the model should continue with the output stream. This "single-stream" format forces the model to implement the target computation as an in-context online process over one evolving buffer, rather than relying on external task selectors or separate invocations. In particular, task identity (when present) must be inferred from tokens within the prefix itself, and the intended next-token rule is uniquely determined for every prefix, preserving condition (A) while enabling controlled studies of conditional and compositional programs.

Work packages each run along an axis of controlled difficulty. We vary task and model complexity along three axes, each designed to increase difficulty while preserving a scale-invariant target program and a uniquely specified continuation.

- **WP1: Program (PDT) complexity.** We vary the structure of the target program while maintaining a single deterministic next-token rule. This axis tests is central to in-context curriculum learning: in particular, does the need to select and compose subroutines increase or decrease the tendency to learn horizon independent short-cuts?

- *Single PDT (context-free prompts):* the prompt specifies a single fixed computation (e.g. a fixed operation or grammar), with no task-context tokens. This isolates length generalization of a single update rule.
- *Conditional PDTs (contextual prompts):* the prompt contains task information (e.g. operation symbols) that selects between a finite family of PDT subroutines. Formally, the target remains a single deterministic program with branching logic conditioned on the prefix, which captures algorithm selection in-context without introducing ambiguity.
- *Compositions of PDT subroutines:* the target program composes multiple PDT-like subroutines within one prompt (e.g. sequential compositions such as "apply A then B"). We use restricted composition patterns that remain implementable with a stack (e.g. bounded-depth composition or syntactically marked subroutine boundaries), so that scale-invariant solutions remain available and interpretable.

- **WP2: Time (computation) complexity.** We vary whether the model must learn a computation schedule in addition to the state update logic. This axis tests whether or not learning variable computation per token increases or decreases the tendency to learn horizon independent short-cuts.
  - *Synchronous transduction:* one input symbol is consumed and one output symbol is produced per step. This isolates length dependence in the update rule without confounding variable-time computation.
  - *Asynchronous / variable-time transduction:* the model is permitted to emit intermediate symbols before producing the final output stream, corresponding to a learned computation schedule or halting rule. A convenient formalization is to reserve a subset of output tokens for a "scratchpad" channel (or to allow a delimiter-separated intermediate segment) while keeping the overall interaction single-stream. This setting isolates whether allowing additional internal steps mitigates or exacerbates length-coupled behavior, and provides an abstract analogue of learning *how long to compute* in chain-of-thought-style behavior.

- **WP3: Memory (representation/storage) complexity.** We vary how prefix information is stored, separating failures of update logic from failures of memory fidelity. This axis analyzes whether length generalization can fail even when storage is perfect, and whether or not failures worsen or qualitatively change when memory storage and control are entangled.
  - *Explicit external stack (baseline):* the controller outputs a distribution over stack actions (push/pop/no-op) and output symbols. This factors out storage capacity/fidelity and allows the learned controller to be interpreted as a program, isolating optimization bias over update rules.
  - *Learned internal storage (relaxed setting):* we remove the external stack and require the model to store task-relevant state in its activations. To keep this tractable, we focus on the simplest tasks (e.g. reverse or bounded-depth bracket matching) and minimal architectures (e.g. a single attention layer with constrained width), allowing us to study how length generalization degrades when storage and control are entangled. In such a setting, a prefix such as $x_1 x_2 \ldots x_n$ = requires the model to implicitly encode a representation of the necessary state (e.g. a pointer or summary) that remains stable as n grows; comparing this regime to the explicit-stack baseline isolates which length failures are fundamentally storage-induced versus arising from the learned update rule itself.

Taken together, these work packages define a controlled hierarchy of settings in which we can (i) establish provable limits and failure modes in the most interpretable regime, (ii) study how conditionality and composition alter the emergence of length-coupled heuristics, and (iii) test which mechanisms persist when computation becomes variable-time or when storage must be learned rather than provided.

3 General setup and assumptions

Alphabet and embeddings. We consider a finite alphabet Σ consisting of data symbols (e.g. $\{0,1\}$), operation or delimiter symbols (e.g. $\{+,-\}$), an equals symbol $=$, and an end-of-sequence symbol `EOS`. Each symbol $x \in \Sigma$ is associated with a fixed embedding vector $\boldsymbol{x} = E(x) \in \mathbb{R}^d$. A sequence of $T \in \mathbb{N}$ symbols is denoted $x_{1:T} \in \Sigma^T$, where x_t denotes the tth symbol in sequence. The embedding of each symbol in the sequence is denoted $\boldsymbol{x}_t = E(x_t) \in \mathbb{R}^d$, thus the embedding of the entire sequence can be represented as a matrix $\boldsymbol{x}_{1:T} \in \mathbb{R}^{d\times T}$. Here, for ease and unless stated otherwise, we assume a one-hot embedding.

Training and test data. We assume each sequence $x_{1:T} \in \Sigma^T$ contains exactly one occurrence of the equals symbol at position $\tau \in [T]$ and exactly one occurrence of the end-of-sequence symbol at the final position. Accordingly, each sequence decomposes into a *prompt* $x_{1:\tau}$ and a *target continuation* $x_{\tau+1:T}$. During training, the full sequence is processed sequentially via teacher forcing. At inference time, only the prompt $x_{1:\tau}$ is provided, and the model generates a continuation autoregressively.

In-context prediction and training loss. For fixed parameters θ, the model defines an in-context next-token predictor

$$p_\theta(\cdot \mid x_{1:t}) \in \Delta(\Sigma),$$

giving a conditional distribution over the next symbol given the prefix. During training, the loss is defined only on the target continuation,

$$\ell(\theta; x_{1:T}) \;:=\; \frac{1}{T-\tau}\sum_{t=\tau}^{T-1} -\log p_\theta(x_{t+1} \mid x_{1:t}),$$

corresponding to the average cross-entropy loss. No loss is defined for $t < \tau$, and the predictor is always conditioned on the true prefix symbols.

Inference and evaluation. At inference time, the model generates tokens autoregressively. Given a prompt $x_{1:\tau}$, for $t \geq \tau$ the model outputs

$$p_\theta(\cdot \mid x_{1:\tau}, \hat{x}_{\tau+1:t}) \in \Delta(\Sigma), \qquad \hat{x}_{t+1} = \arg\max_{\sigma\in\Sigma} p_\theta(\sigma \mid x_{1:\tau}, \hat{x}_{\tau+1:t}).$$

Generation terminates when $\hat{x}_t = \texttt{EOS}$; we denote the resulting termination time by $\hat{T}$. We evaluate performance using both exact-match error,

$$\mathbf{1}\big(\hat{x}_{\tau+1:\hat{T}} \neq x_{\tau+1:T}\big),$$

and token-wise error,

$$\sum_{t=\tau+1}^{\max\{T,\hat{T}\}} \mathbf{1}\big(\hat{x}_t \neq x_t\big),$$

where positions beyond the shorter sequence are counted as errors.

4 General details on setup

4.1 Memory controller

At each time step the memory controller takes in the the current input, state and stack read (more on this later) and outputs two probability distributions encoded as vectors.

Controller inputs:

- **Input symbol / embedding:** without loss of generality, we consider a buffer alphabet $[\Sigma] = \{1, 2...\Sigma\}$ for some $\Sigma \in \mathbb{N}$. Each buffer symbol is represented as a one hot vector, more specifically, $E(\sigma) \in \{0,1\}^{\Sigma}$ where $E(\sigma)_j = \mathbb{1}(j = \gamma)$ for all $j \in [\Sigma]$. Then the input symbol at time $t \in \mathbb{N}$ is denoted $x(t) \in [\Sigma]$ and its one hot embedding as $\boldsymbol{x}(t) \in \{0,1\}^{\Sigma}$.
- **State vector / distribution:** in order to simulate a PDT, the controller can be in one of Q states (after initialization). The state vector of the controller at time t is is $\boldsymbol{q}(t) \in \Delta([Q])$, and represents a distribution over the possible states the model can be in. The initial state is zero, i.e., $\boldsymbol{q}(1) = \mathbf{0}_Q$. When $\boldsymbol{q}(t)$ is a single point mass we recover the deterministic case.
- **Top of stack vector / distribution:** the controller can read the contents of the top of the stack, see below for more information. In particular, $\boldsymbol{r}(t) \in \Delta([\Gamma])$ represents the distribution over symbols, read from the top of the stack, where there are $\Gamma \in \mathbb{N}$ stack symbols, at time $t \in \mathbb{N}$. See the next subsection for more details.

Controller outputs:

- **Memory action:** suppose there are Γ distinct symbols that can be stored on the stack, including a symbol which denotes the empty stack. Without loss of generality we let the empty stack symbol be Γ. Then there are $\Gamma + 1$ distinct memory operations that the controller can signal to be performed:
$$\{\texttt{PUSH}(\gamma) : \gamma \in [\Gamma - 1]\} \cup \{\texttt{POP}, \texttt{NOOP}\}$$
Then the vector over memory actions is at time t is denoted $\boldsymbol{a}(t) \in \Delta([\Gamma + 1])$ where $a_\gamma(t)$ is either i) the probability of pushing the symbol γ onto the stack for $\gamma \in [\Gamma - 1]$, ii) the probability of popping from the stack if $\gamma = \Gamma$ or iii) is the probability of performing a do nothing operation if $\gamma = \Gamma + 1$.
- **Buffer action:** at each time step the controller signals to write / append an element to the buffer. Recalling that the set of $[\Sigma]$

The controller itself then can be interpreted as a sequence to sequence model, mapping sequences of input tokens to a sequence of distributions over tokens.

4.2 External Stack Memory

Stack alphabet: Without loss of generality we consider a stack alphabet $[\Gamma] = \{1, 2...\Gamma\}$ for some $\Gamma \in \mathbb{N}$. Moreover, allocate Γ as the *empty stack* symbol: this symbol cannot be pushed onto the stack and popping it has no effect. Each stack symbol is represented as a one hot vector, more specifically $E(\gamma) \in \{0,1\}^{\Gamma}$ where $E(\gamma)_j = \mathbb{1}(j = \gamma)$ for all $j \in [\Gamma]$.

Stack data structure and operations: we consider a finite stack of depth $K \in \mathbb{N}$. The stack at time $t \in \mathbb{N}$ we represent as a matrix $\boldsymbol{M}(t) \in \mathbb{R}^{K \times \Gamma}$. The kth row of $\boldsymbol{M}(t)$, $\boldsymbol{m}_k(t) \in \Delta([\Gamma])$, holds a probability mass function over the possible stack symbols stored at the kth position of the stack at time t. Note, when $\boldsymbol{m}_k(t)$ is one-hot then this setup collapses to the deterministic setting of storing a the embedding of a single stack symbol. At initialization $\boldsymbol{m}_k(1) = [0, 0, ...1] = \boldsymbol{e}_\Gamma$, i.e., all stack slots with probability one are empty. During each time step a number of operations can take place

Reading / sensing the top of the stack: it is standard that the memory controller is able to read the last value pushed to the stack, or equivalently, the object at the top of the stack. Mirroring this, we let $\boldsymbol{r}(t) \in \Delta([\Gamma])$ denote the distribution over the value stored at the top of the stack at time t. In order to simplify matters we always define $\boldsymbol{m}_1(t)$ as the top of the stack at time t, in particular

$$\boldsymbol{r}(t) = \boldsymbol{m}_1(t). \tag{1}$$

Smaller indices correspond to being closer to the top of the stack, larger to being nearer the bottom. Whenever we push all the information moves down the stack (up the indices) and whenever we pop information moves up the stack (down the indices).

The stack during training: soft stack implementation. During training, at a timestep t the controller outputs a distribution $\boldsymbol{a}(t)$ over possible stack actions as described above. To recursively update the stack we compute the expected distribution over symbols with respect to the distribution of possible actions,

$$\boldsymbol{m}_k(t+1) = \begin{cases} \left(\sum_{\gamma=1}^{\Gamma-1} a_\gamma(t)\boldsymbol{e}_\gamma\right) + a_{noop}(t)\boldsymbol{m}_1(t) + a_{pop}(t)\boldsymbol{m}_2(t), & k = 1, \\ a_{push}(t)\boldsymbol{m}_{k-1}(t) + a_{noop}(t)\boldsymbol{m}_k(t) + a_{pop}(t)\boldsymbol{m}_{k+1}(t), & k \in [2, K-1], \\ a_{push}(t)\boldsymbol{m}_{k-1}(t) + a_{noop}(t)\boldsymbol{m}_k(t) + a_{pop}(t)\boldsymbol{e}_\Gamma, & k = K. \end{cases}$$

where here $a_{push}(t) = \sum_{\gamma=1}^{\Gamma-1} a_\gamma(t)$, $a_{noop}(t) = a_{\Gamma+1}(t)$ and $a_{pop}(t) = a_\Gamma(t)$ are introduced for clarity. A few remarks are in order.

- **Rows continue to lie on the simplex:** if $\boldsymbol{a}(t) \in \Delta([\Gamma+1])$, and $\boldsymbol{m}_k(t) \in \Delta([\Gamma])$, then under this recursive update rule $\boldsymbol{m}_k(t+1) \in \Delta([\Gamma])$.
- **Finite truncation depth:** if we push to many elements onto the stack then information is lost as it is eventually pushed off the end of the stack and cannot be recovered by popping (note for $k = K$ we weight by the empty symbol). Generally we assume that K is sufficiently large so as to ensure that this is not a problem.
- **Backpropagation:** Note that these operations are linear, therefore back-propagating through these soft-stack operations is trivial.

We can actually write this update in matrix form by introducing a downshift and an upshift operator. To this end let $\boldsymbol{U}, \boldsymbol{D} \in \{0,1\}^{K \times K}$ be defined as

$$\boldsymbol{U} = \begin{bmatrix} \mathbf{0}_{K-1} & \mathbf{I}_{K-1} \\ 0 & \mathbf{0}_{K-1}^T \end{bmatrix}, \quad \boldsymbol{D} = \begin{bmatrix} 0 & \mathbf{0}_{K-1}^T \\ \mathbf{I}_{K-1} & \mathbf{0}_{K-1} \end{bmatrix}.$$

Then we can write the update rule for the entire stack as

$$\boldsymbol{M}(t+1) = \left(a_{noop}(t)\boldsymbol{I} + a_{push}(t)\boldsymbol{D} + a_{pop}(t)\boldsymbol{U}\right)\boldsymbol{M}(t) + \boldsymbol{e}_1\left(\sum_{\gamma=1}^{\Gamma-1} a_\gamma(t)\boldsymbol{e}_\gamma\right)^T + a_{pop}(t)\boldsymbol{e}_K\boldsymbol{e}_\Gamma^T \tag{2}$$

Algebraic properties of the shift operators. Recall the stack update rule

$$\boldsymbol{M}(t+1) = \boldsymbol{A}(t)\boldsymbol{M}(t) + \boldsymbol{B}(t), \tag{3}$$

where

$$\boldsymbol{A}(t) = a_{\text{noop}}(t)\boldsymbol{I} + a_{\text{push}}(t)\boldsymbol{D} + a_{\text{pop}}(t)\boldsymbol{U}, \tag{4}$$

$$\boldsymbol{B}(t) = \boldsymbol{e}_1\boldsymbol{s}(t)^T + a_{\text{pop}}(t)\boldsymbol{e}_K\boldsymbol{e}_\Gamma^T, \quad \boldsymbol{s}(t) = \sum_{\gamma=1}^{\Gamma-1} a_\gamma(t)\boldsymbol{e}_\gamma. \tag{5}$$

The shift matrices $\boldsymbol{D}, \boldsymbol{U} \in \mathbb{R}^{K\times K}$ satisfy several useful structural identities. First, both operators are nilpotent of order K,

$$\boldsymbol{D}^K = \boldsymbol{U}^K = \boldsymbol{0}, \tag{6}$$

reflecting the finite depth of the stack. Second, their compositions satisfy

$$\boldsymbol{U}\boldsymbol{D} = \boldsymbol{I} - \boldsymbol{e}_1\boldsymbol{e}_1^T, \ \boldsymbol{D}\boldsymbol{U} = \boldsymbol{I} - \boldsymbol{e}_K\boldsymbol{e}_K^T, \tag{7}$$

showing that push and pop operations act as inverses away from the stack boundaries. The shift operators also satisfy the boundary annihilation relations

$$\boldsymbol{e}_1^T\boldsymbol{D} = \boldsymbol{0}, \ \boldsymbol{e}_K^T\boldsymbol{U} = \boldsymbol{0}, \ \boldsymbol{D}\boldsymbol{e}_K = \boldsymbol{0}, \ \boldsymbol{U}\boldsymbol{e}_1 = \boldsymbol{0}. \tag{8}$$

More generally, for any integer $i \geq 0$, the powers of the shift operators satisfy

$$(\boldsymbol{D}^i\boldsymbol{M}(t))_k = \begin{cases} \boldsymbol{m}_{k-i}(t) & k > i, \\ \boldsymbol{0} & k \leq i, \end{cases} \tag{9}$$

$$(\boldsymbol{U}^i\boldsymbol{M}(t))_k = \begin{cases} \boldsymbol{m}_{k+i}(t) & k \leq K-i, \\ \boldsymbol{0} & k > K-i. \end{cases} \tag{10}$$

These identities imply that the stack evolution defines an affine linear dynamical system driven by boundary injection terms. In particular, unrolling the recursion yields

$$\boldsymbol{M}(t) = \left(\prod_{s=1}^{t-1}\boldsymbol{A}(s)\right)\boldsymbol{M}(1) + \sum_{s=1}^{t-1}\left(\prod_{u=s+1}^{t-1}\boldsymbol{A}(u)\right)\boldsymbol{B}(s). \tag{11}$$

Finally, since $\boldsymbol{A}(t)$ is a convex combination of partial shift operators, it defines a stable transport operator over stack depth. In particular, it preserves non-negativity and boundedness of the stack contents.

Model structure: the model outputs at each time step three distributions: one over the memory action, another over the state and a third over the buffer action. Let $\boldsymbol{z}(t) = [\boldsymbol{m}_1(t)^T, \boldsymbol{x}(t)^T, \boldsymbol{q}(t)^T]^T$. Then we define

$$\begin{aligned}
\boldsymbol{a}(t+1) &= \texttt{Softmax}(\hat{\boldsymbol{a}}(t+1)), \quad \hat{\boldsymbol{a}}(t+1) = F^{(a)}(\boldsymbol{z}(t);\boldsymbol{\theta}) \\
\boldsymbol{b}(t+1) &= \texttt{Softmax}(\hat{\boldsymbol{b}}(t+1)), \quad \hat{\boldsymbol{b}}(t+1) = F^{(b)}(\boldsymbol{z}(t);\boldsymbol{\theta}) \\
\boldsymbol{q}(t+1) &= \texttt{Softmax}(\hat{\boldsymbol{q}}(t+1)), \quad \hat{\boldsymbol{q}}(t+1) = F^{(q)}(\boldsymbol{z}(t);\boldsymbol{\theta}).
\end{aligned}$$

Goals / things want to try and compute

- High level goal: prove the emergence of length coupled programs (e.g., find parameters which result in an implicit program which does not generalize well to length) during training. Quantify length generalization error in terms of test time sequence length versus train time sequence.
- Probably over-ambitious target: find a simple setting where show GD probably converges to a solution which does not length generalize: e.g., uses state vector as a counter, or limited by growth in entropy of stack contents near the bottom of the stack.
- Easier things to compute: fix the memory policy, e.g., even if we fix $\boldsymbol{a}(t)$ as the correct memory policy in time say, what happens then?

4.3 Deriving expressions for the gradients: generic model

For training we consider the teacher forcing setup. To this end consider a dataset of n sequence to sequence maps: more specifically, let $(\boldsymbol{x}(t), \boldsymbol{y}(t))_{t\in[T_i]}$ be a sequence pair of the dataset. Here, $\boldsymbol{x}(t)$ denotes the input token at position t of the sequence and $\boldsymbol{y}$ can be interpreted as the target distribution over the token at position $t+1$. Unless stated otherwise We let $\tau_{\boldsymbol{x}}$ denote the position in the sequence where we wish to start making accurate predictions (in practice this would typically be the end of the prompt part of the sequence). We denote the full cross entropy loss on the sequence as

$$\ell((\boldsymbol{x}, \boldsymbol{y}); \boldsymbol{\theta}) = \frac{1}{T_{\boldsymbol{x}} - \tau_{\boldsymbol{x}}} \sum_{t=\tau_{\boldsymbol{x}}}^{T_{\boldsymbol{x}}-1} \texttt{CE}(\boldsymbol{b}(t; \boldsymbol{\theta}), \boldsymbol{y}(t))$$

where $\boldsymbol{b}(t)$ is the prediction of the distribution over the next token made by our model. We parameterize these predictions using the vector of parameters $\boldsymbol{\theta}$. We then average over the this loss over the training set $\mathcal{D}$,

$$L(\boldsymbol{\theta}) = \frac{1}{|\mathcal{D}|} \sum_{(\boldsymbol{x},\boldsymbol{y})\in\mathcal{D}} \ell((\boldsymbol{x}, \boldsymbol{y}); \boldsymbol{\theta})$$

For typographical clarity going forward we omit the dependence on $\boldsymbol{\theta}$ as implicit. Letting θ denote an element of $\boldsymbol{\theta}$ then we compute the gradient of the full loss as

$$\frac{\partial L}{\partial \theta} = \frac{1}{|\mathcal{D}|} \sum_{(\boldsymbol{x},\boldsymbol{y})\in\mathcal{D}} \frac{\partial \ell(\boldsymbol{x}, \boldsymbol{y})}{\partial \theta}.$$

We proceed to focus on a single sequence to sequence cross entropy term, for typographical ease we further drop the explicit dependence on the sequence pair and let $\ell = \ell(\boldsymbol{x}, \boldsymbol{y})$ for some pair $(\boldsymbol{x}, \boldsymbol{y})$. We first recall some useful identities for differentiating through softmax and the cross-entropy loss. Let $\hat{\boldsymbol{p}} \in \mathbb{R}^d$, $\boldsymbol{p} = \texttt{Softmax}(\hat{\boldsymbol{p}})$. For typographical ease let $\delta_{ij} = \mathbb{1}(j = i)$. Then we have the following identities,

$$\frac{\partial p_l}{\partial \hat{p}_j} = p_l(\delta_{lj} - p_j), \quad \frac{\partial \texttt{CE}(\boldsymbol{p}, \hat{\boldsymbol{y}})}{\partial \hat{p}_j} = p_j - y_j.$$

Now let θ be a scalar parameter or element of $\boldsymbol{\theta}$. Then

$$\begin{aligned}
\frac{\partial \ell}{\partial \theta} &= \frac{1}{T-\tau}\sum_{t=\tau}^{T-1} \frac{\partial \texttt{CE}(\boldsymbol{b}(t;\boldsymbol{\theta}), \boldsymbol{y}(t))}{\partial \theta} \\
&= \frac{1}{T-\tau}\sum_{t=\tau}^{T-1}\sum_{j=1}^{\Sigma} \frac{\partial \texttt{CE}(\boldsymbol{b}(t;\boldsymbol{\theta}), \boldsymbol{y}(t))}{\partial \hat{b}_j(t)} \left(\frac{\partial \hat{b}_j(t)}{\partial \theta} + \sum_{k=1}^{d+\Gamma+Q} \frac{\partial \hat{b}_j(t)}{\partial z_k(t)} \frac{\partial z_k(t)}{\partial \theta} \right) \\
&= \frac{1}{T-\tau}\sum_{t=\tau}^{T-1}\sum_{j=1}^{\Sigma} \varepsilon_j(t) \left(\frac{\partial \hat{b}_j(t)}{\partial \theta} + \sum_{k=1}^{d+\Gamma+Q} \frac{\partial \hat{b}_j(t)}{\partial z_k(t)} \frac{\partial z_k(t)}{\partial \theta} \right)
\end{aligned}$$

We can also vectorize this as

$$\frac{\partial \ell}{\partial \theta} = \frac{1}{T-\tau}\sum_{t=\tau}^{T-1} \boldsymbol{\varepsilon}(t)^T \left(\frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \theta} + \frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{z}(t)} \frac{\partial \boldsymbol{z}(t)}{\partial \theta} \right),$$

where $\frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{z}(t)} \in \mathbb{R}^{\Sigma \times (d+\Gamma+Q)}$,

$$\left[\frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{z}(t)} \right]_{jk} = \frac{\partial \hat{b}_j(t)}{\partial z_k(t)}.$$

Some remarks are as follows.

- The term $\varepsilon_j(t) = b_j(t) - y_j(t)$ can be interpreted as an error term.
- The term $\frac{\partial \hat{b}_j(t)}{\partial \theta}$ is the direct partial differential (treats $\boldsymbol{z}(t)$ as fixed) and depends on the way $F^{(b)}$ has been parameterized by θ. Similarly the terms $\frac{\partial \hat{b}_j(t)}{\partial z_k(t)}$ also are defined by the specifics of the parameterization of $F^{(b)}$.
- The term $\frac{\partial z_k(t)}{\partial \theta}$ is where the recurrence comes in. Note that as the inputs do not depend on θ this reduces to understanding the two partial differentials $\frac{\partial m_{1k}(t)}{\partial \theta}$ (propagation through the stack) and $\frac{\partial q_k(t)}{\partial \theta}$.

As a result, and given that the input tokens are independent of the model parameters then the key terms we turn our attention to are

$$\frac{\partial \boldsymbol{z}(t)}{\partial \theta} = \begin{bmatrix} \frac{\partial \boldsymbol{m}_1(t)}{\partial} \\ \mathbf{0}_d \\ \frac{\partial \boldsymbol{q}(t)}{\partial \theta}. \end{bmatrix}$$

This implies we can rewrite the gradient of the loss with the following block structure,

$$\frac{\partial \ell}{\partial \theta} = \frac{1}{T-\tau}\sum_{t=\tau}^{T-1} \boldsymbol{\varepsilon}(t)^T \left(\frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \theta} + \frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{m}_1(t)} \frac{\partial \boldsymbol{m}_1(t)}{\partial \theta} + + \frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{q}(t)} \frac{\partial \boldsymbol{q}(t)}{\partial \theta} \right) \tag{12}$$

Gradient of the state. Recall that the logits are written as $\hat{\boldsymbol{q}}(t) = F^{(q)}(\boldsymbol{z}(t-1);\boldsymbol{\theta})$. Therefore

$$\frac{\partial \hat{q}_j(t)}{\partial \theta} = \frac{\partial F_j^{(q)}(\boldsymbol{z}(t-1);\boldsymbol{\theta})}{\partial \theta} + \frac{\partial F_j^{(q)}(\boldsymbol{z}(t-1);\boldsymbol{\theta})}{\partial z_k(t-1)} \frac{\partial z_k(t-1)}{\partial \theta}$$

or in vectorized form

$$\frac{\partial \hat{\boldsymbol{q}}(t)}{\partial \theta} = \frac{\partial F^{(q)}(\boldsymbol{z}(t-1);\boldsymbol{\theta})}{\partial \theta} + \frac{\partial F^{(q)}(\boldsymbol{z}(t-1);\boldsymbol{\theta})}{\partial \boldsymbol{z}(t-1)} \frac{\partial \boldsymbol{z}(t-1)}{\partial \theta}$$

As $\boldsymbol{q}(t) = \texttt{Softmax}(\hat{\boldsymbol{q}}(t))$ then

$$\frac{\partial \boldsymbol{q}(t)}{\partial \hat{\boldsymbol{q}}(t)} = \operatorname{diag}(\boldsymbol{q}(t)) - \boldsymbol{q}(t)\boldsymbol{q}(t)^T =: \boldsymbol{J}_{sm}^{(q)}(t),$$

which therefore implies by the chain rule

$$\frac{\partial \boldsymbol{q}(t)}{\partial \theta} = \boldsymbol{J}_{sm}^{(q)}(t) \left(\frac{\partial F^{(q)}(t-1)}{\partial \theta} + \frac{\partial F^{(q)}(t-1)}{\partial \boldsymbol{z}(t-1)} \frac{\partial \boldsymbol{z}(t-1)}{\partial \theta} \right)$$

where we use the notation $F^{(q)}(t) = F^{(q)}(\boldsymbol{z}(t);\boldsymbol{\theta}) = \hat{\boldsymbol{b}}(t)$ for typographical convenience. Moreover, we can simplify this further as

$$\frac{\partial \boldsymbol{q}(t+1)}{\partial \theta} = \boldsymbol{J}_{sm}^{(q)}(t+1) \left(\frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \theta} + \frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{m}_1(t)} \frac{\partial \boldsymbol{m}_1(t)}{\partial \theta} + \frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{q}(t)} \frac{\partial \boldsymbol{q}(t)}{\partial \theta} \right). \tag{13}$$

Gradient of the top of the stack. First we recall the top-of-stack update,

$$\boldsymbol{m}_1(t+1) = \boldsymbol{s}(t) + a_{\text{noop}}(t)\boldsymbol{m}_1(t) + a_{\text{pop}}(t)\boldsymbol{m}_2(t).$$

Using the product rule it follows that

$$\frac{\partial \boldsymbol{m}_1(t+1)}{\partial \theta} = \frac{\partial \boldsymbol{s}(t)}{\partial \theta} + a_{\text{noop}}(t)\frac{\partial \boldsymbol{m}_1(t)}{\partial \theta} + \frac{\partial a_{\text{noop}}(t)}{\partial \theta}\boldsymbol{m}_1(t) + a_{\text{pop}}(t)\frac{\partial \boldsymbol{m}_2(t)}{\partial \theta} + \frac{\partial a_{\text{pop}}(t)}{\partial \theta}\boldsymbol{m}_2(t).$$

Observing the identities

$$\frac{\partial \boldsymbol{s}(t)}{\partial \theta} = \boldsymbol{V}\frac{\partial \boldsymbol{a}(t)}{\partial \theta}, \qquad \frac{\partial a_{\text{pop}}(t)}{\partial \theta} = \boldsymbol{e}_\Gamma^\top \frac{\partial \boldsymbol{a}(t)}{\partial \theta}, \qquad \frac{\partial a_{\text{noop}}(t)}{\partial \theta} = \boldsymbol{e}_{\Gamma+1}^\top \frac{\partial \boldsymbol{a}(t)}{\partial \theta}, \tag{14}$$

where $\boldsymbol{V} = [\boldsymbol{e}_1, \ldots, \boldsymbol{e}_{\Gamma-1}, \boldsymbol{0}, \boldsymbol{0}] \in \mathbb{R}^{\Gamma \times (\Gamma+1)}$, so that $\boldsymbol{s}(t) = \boldsymbol{V}\boldsymbol{a}(t)$, $a_{\text{pop}}(t) = \boldsymbol{e}_\Gamma^\top \boldsymbol{a}(t)$, $a_{\text{noop}}(t) = \boldsymbol{e}_{\Gamma+1}^\top \boldsymbol{a}(t)$, then we conclude that

$$\frac{\partial \boldsymbol{m}_1(t+1)}{\partial \theta} = \boldsymbol{V}\frac{\partial \boldsymbol{a}(t)}{\partial \theta} + a_{\text{noop}}(t)\frac{\partial \boldsymbol{m}_1(t)}{\partial \theta} + a_{\text{pop}}(t)\frac{\partial \boldsymbol{m}_2(t)}{\partial \theta} + \left(\boldsymbol{e}_{\Gamma+1}^\top \tfrac{\partial \boldsymbol{a}(t)}{\partial \theta}\right)\boldsymbol{m}_1(t) + \left(\boldsymbol{e}_\Gamma^\top \tfrac{\partial \boldsymbol{a}(t)}{\partial \theta}\right)\boldsymbol{m}_2(t)$$

Summary of gradient First we substitute the partial derivative of the state into the expression for the partial derivative of the loss. Recalling that

$$\frac{\partial \ell}{\partial \theta} = \frac{1}{T-\tau}\sum_{t=\tau}^{T-1} \boldsymbol{\varepsilon}(t)^T \left(\frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \theta} + \frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{m}_1(t)} \frac{\partial \boldsymbol{m}_1(t)}{\partial \theta} + \frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{q}(t)} \frac{\partial \boldsymbol{q}(t)}{\partial \theta} \right),$$

$$\frac{\partial \boldsymbol{q}(t+1)}{\partial \theta} = \boldsymbol{J}_{sm}^{(q)}(t+1) \left(\frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \theta} + \frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{m}_1(t)} \frac{\partial \boldsymbol{m}_1(t)}{\partial \theta} + \frac{\partial \hat{\boldsymbol{b}}(t)}{\partial \boldsymbol{q}(t)} \frac{\partial \boldsymbol{q}(t)}{\partial \theta} \right),$$